Deployment: the final waterfall

Steve Loughran, Sally Blue Kaneshiro, Mark Newsome

November 20001

The challenge of deployment

In the summer of 2001, the Evergreen project delivered photographic image processing *Web Services* as a commercial service for an external customer. One of the two Web Services supported the upload of images and the download of the images at various resolutions, and was derived from the HPPhoto web site code. The other accepted descriptions of pages in Scalable Vector Graphics format (SVG), and rendered these into 72 dpi preview images or full 300 dpi images for printing onto the customer's farm of Indigo printers in a different state. Our service was co-located in San Jose, with a beta release of the system hosted on our own site.

As in any software project, many things interfered with the smooth execution of the plan. What surprised us was how many of the problems were not defects in the code, as much as configuration problems in the final system, network problems and other deployment issues. These issues not only made delivery harder, they continued to be a source of support calls after the system went live.

Here are some of the problems we encountered:

- An eight-hour time difference between the rendering service and the network file store sometimes caused the housekeeping routine to delete files as they were created.
- Failure to configure accounts and permissions prevented services from saving files to the store.
- Not all the customer's fonts were on all systems, leading to inconsistent results.
- The configuration files were incorrect; for example, the IP addresses of dependent systems were wrong.
- A needed COM object 'disappeared' from one machine, for unknown reasons.
- An outage on an upstream DNS server stopped parts of the cluster talking to other parts, raising a `java.io.NoRouteToHostException`, which operations took to mean a Java problem, rather than a networking issue.

Many of these problems could be blamed on the complexity of bringing up a new system from scratch; the HPPhoto based asset store service had thirty-five pages of installation file talking the installer through all the dialog boxes and commands needed to install it. However, even the Java-based SVG rendering service had a six-page installation guide, covering the installation of Java, the Application Server, the Win32 components and the user account configuration. With so much configuration and installation, it is inevitable that things would go wrong, and go wrong they did.

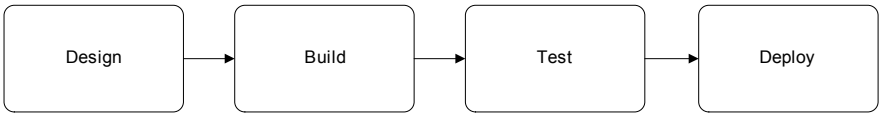
The fundamental problems

The underlying causes of these problems are fourfold:

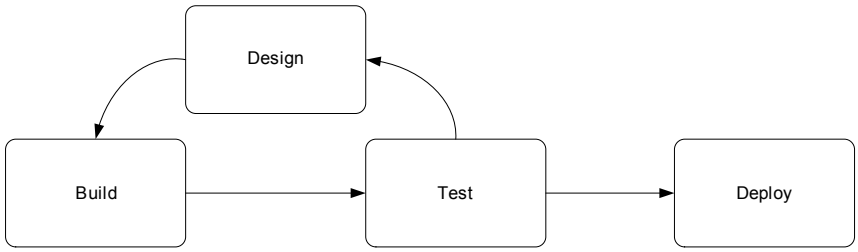
1. Complex, multi-machine clusters are complicated to configure, and errors creep into the configuration.
2. Manual installation processes are slow and unreliable.
3. The needs of operations were not taken into account in the design of the application.
4. The lessons of each deployment cycle were not fed back into the software.

To address these problems we need to adopt a development process that recognizes that deployment and operations are critical problem areas. Yet all modern software development lifecycles treat deployment as an afterthought.

The original process for writing software is called *The Waterfall Process*, as the software project moves from one stage to another in a linear and one-way manner:



This process is now strongly disparaged, as it is too inflexible. Instead, iterative or evolutionary processes iterate through the design, build and test cycles, redesigning an application based on the experience of each iteration, and potentially releasing a version of the system every cycle:



Notice, however, that even in such an iterative system, deployment is still an afterthought in which a system is *handed off* to operations. It is our premise that to improve the time to market and reduce the operational cost of web services, we need a new process.

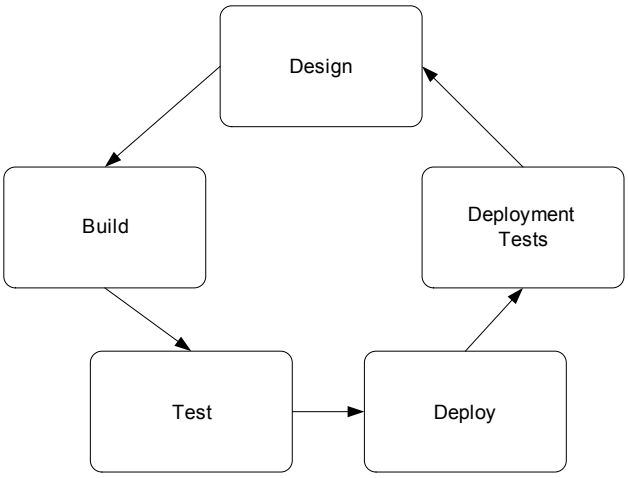
Introducing *Continuous Deployment*

The name for our new process is *Continuous Deployment,* extending the notion of *Continuous Integration* that automates the build and test stages of a software project. In continuous deployment, the act of deploying the software is fully automated, and executed automatically every night. This keeps the Web Service is up to date, enabling the external users of the service to develop against the most recent version.

Automating deployment can eliminate some configuration problems, but not all of them. Our process addresses this by viewing any configuration problem exactly as if it were any other software defect: a configuration defect should have a bug report filed against it and a test created to verify the defect. After the test is written, the configuration can be corrected. Yet the test is retained, and rerun after every future deployment. This ensures that the same configuration problem will not re-occur –or if it does, the problem will be detected immediately. The presence of a bug report for every problem also ensures that the fix for every configuration defect is easily identifiable.

It is this notion: *configuration problems are just another testable software defect*, which is new, along with, a process and deployment stage that can be included in existing processes such as the Rational Unified Process.

The resulting lifecycle brings deployment and deployment testing into the main iterative cycle, as shown here.



Testing some of the configuration defects enumerated earlier, server time zone differences are tested by creating a file on the filestore and comparing the timestamp with the local clock. Access rights are verified by creating files, and IP address configurations validated by probing for services at the configured addresses.
More complex tests load a COM object, or validate rendered images against a bitmap gold standard copy –this can test for needed fonts.

These tests can not only be used immediately after deployment, they can be invoked on the running servers, so that monitoring systems, including OpenView, can probe a functional service and verify that it is healthy. When a server is unhealthy, the detailed tests make it easy to track down where the fundamental problem lies, reducing downtime. This aids operations, but it also benefits the development team, who get called in less.

Status

We evolved this process as our project proceeded, although we could only automate deployment to the local staging sites; security issues forced us to retain a manual deployment process for the production system.

As we added more tests to the system, it became easier to validate even manual deployment; instead of debugging the process by redoing every step, the health check report could pinpoint where the problems lay.

We are using this technique in ongoing projects, and evangelizing it to the software development community as a whole. A co-authored book on the Java build tool Ant, has a

chapter on it, and we have contributed our health tests for the Apache Axis SOAP server to that project. This health check web page to validate common configuration problems has become the first port of call for all support issues: whenever a user has an installation problem, they are first asked to check that happyaxis.jsp is happy, and if not, to follow its advice.

We hope, therefore, that aspects of this methodology will become commonplace in all future server-side projects, inside and outside HP.